RigorBench: Benchmarking Engineering Process Discipline in Autonomous AI Coding Agents

Meher Sai Preetam Madiraju
mehersaipreetam@gatech.edu

Meher Bhaskar Madiraju
meherbhaskar.madiraju@gatech.edu

Abstract

Agentic coding harnesses—such as Agent-Skills, Superpowers, and the authors' own reference implementation, Agent-Rigor—are increasingly deployed to augment underlying LLMs for real-world software engineering tasks. Existing benchmarks evaluate these agents almost exclusively on *outcome correctness*: whether generated code passes tests or resolves issues. We argue that this outcome-only lens is insufficient: an agent that arrives at a correct solution through reckless trial-and-error, without planning, verification, or graceful recovery, is fundamentally less reliable than one that follows sound engineering discipline. We introduce RIGORBENCH, an exploratory benchmark designed to measure *process discipline* in AI coding agents. RIGORBENCH evaluates these harnesses across five pillars: PLANNING FIDELITY, VERIFICATION COVERAGE, RECOVERY EFFICIENCY, ABSTENTION QUALITY, and ATOMIC TRANSITION INTEGRITY. A composite RIGORSCORE aggregates these dimensions into a single metric via a weighted sum. We curate an exploratory pilot suite of 100 tasks spanning five categories—Plan-Then-Build, Verify-Or-Die, Doom Loop Gauntlet, Know When to Fold, and Don't Break the Build—and evaluate leading harnesses in a comparative study against baseline coding assistants. Our results show that while tool-enforced frameworks fail to improve baseline process metrics, upfront planning-enforced frameworks (like Agent-Rigor) improve process quality scores by 31% and raise downstream outcome correctness by 30%, providing the first quantitative evidence that *how* agents code matters. We release the full benchmark, scoring rubrics, and trajectory analysis tools as open-source artifacts at https://github.com/MeherBhaskar/RigorBench.

1 Introduction

The rapid maturation of large language models (LLMs) has given rise to a new class of software engineering tool: the *agentic coding harness*. Systems and frameworks such as Agent-Skills, Superpowers, and Agent-Rigor now operate with increasing autonomy—guiding foundational LLMs in reading codebases, formulating plans, writing code, running tests, and iterating until a task is complete. Their capabilities are evaluated by an expanding ecosystem of benchmarks: SWE-bench [Jimenez et al., 2024] for resolving real GitHub issues, HumanEval [Chen et al., 2021] and MBPP [Austin et al., 2021] for function-level synthesis, BigCodeBench [Zhuo et al., 2024] for complex API usage, and domain-specific suites such as Terminal-Bench [Xie et al., 2024] and ProjDevBench [Zhang et al., 2024b] for project-scale development.

These benchmarks share a common evaluation philosophy: they measure *outcomes*. Did the generated code pass the test suite? Was the GitHub issue marked resolved? Does the function return the correct output on the hidden test set? While outcome correctness is a necessary condition for useful code generation, we contend that it is not a sufficient one for *reliable* deployment of autonomous coding agents.

The problem with outcome-only evaluation. Consider two agents solving an identical bug-fix task. Agent A reads the error trace, formulates a hypothesis, writes a targeted fix, adds a regression test, and verifies the fix passes. Agent B tries five different patches in sequence, each time running the test suite and observing failures, until it stumbles upon a patch that makes the tests pass—without understanding *why* the fix works and without adding any verification. Under every existing benchmark, both agents receive the same score. Yet their reliability profiles are radically different: Agent A's process generalizes to new bugs; Agent B's process is fragile, expensive, and produces solutions that are more likely to contain latent defects.

This observation is not new in software engineering. Decades of research on software process maturity—from Humphrey's Personal Software Process [Humphrey, 1989] to the Capability Maturity Model Integration (CMMI) [CMMI Institute, 2018]—have established that *how* software is built is a strong predictor of its long-term quality, maintainability, and cost. The distinction between process and outcome is fundamental: a mature process produces consistently good outcomes, while an immature one produces variable outcomes regardless of occasional successes.

Contributions. We make the following contributions:

1. **We identify and formalize the process discipline gap.**

We survey 12 major AI coding benchmarks and demonstrate that none evaluate the engineering process through which agents arrive at solutions (Section 3).

2. **We introduce RIGORBENCH.** We design the first benchmark that evaluates AI coding agents on engineering process discipline through five measurement pillars, each targeting a distinct dimension of sound software practice (Section 4).

3. **We propose trajectory-based scoring.** Rather than evaluating only final artifacts, RIGORBENCH analyzes the *full execution trajectory* of an agent—every plan, edit, test invocation, error recovery, and commit—to compute process quality scores (Section 4.3).

4. **We demonstrate the value of structured discipline.** Through a comparative evaluation using the `agent-rigor` framework [**?**] (which was co-developed by the co-authors of this work) as a reference implementation, we show that upfront planning-enforced discipline significantly improves both process quality (+31%) and outcome quality (+30%) (Section 6), while merely tool-enforced harnesses do not.

5. **We release an open benchmark suite.** We release a suite of 100 tasks across 5 categories, complete with scoring rubrics, trajectory analysis tools, and baseline results, to enable reproducible evaluation of process discipline in future agents (Section 5).

2 Related Work

Code generation benchmarks. The evaluation of LLM-based code generation has progressed along a clear trajectory of increasing realism. HumanEval [Chen et al., 2021] and MBPP [Austin et al., 2021] established function-level benchmarks with unit-test-based pass rates. BigCodeBench [Zhuo et al., 2024] extended this to multi-library compositions. SWE-bench [Jimenez et al., 2024] introduced real-world GitHub issue resolution, requiring agents to navigate complex codebases and produce patches that pass existing test suites. LiveCodeBench [Jain et al., 2024] and PeerBench [Cheng et al., 2025] addressed contamination concerns by using temporally filtered competition problems and proctored execution environments. DevBench [Li et al., 2024], ProjDevBench [Zhang et al., 2024b], and ProjectEval [Liu et al., 2025] pushed toward full project-scale development and automated simulation of user acceptance. Terminal-Bench [Xie et al., 2024] evaluates agents in realistic terminal environments. AgentBench [Liu et al., 2023] provides a multi-dimensional evaluation of LLMs as agents across diverse environments.

Despite this rich landscape, every benchmark in this lineage evaluates *what* the agent produces, not *how* it produces it. RIGORBENCH is, to our knowledge, the first benchmark explicitly designed to measure the engineering process.

Agent architectures and self-correction. The ReAct paradigm [Yao et al., 2023] interleaves reasoning and action, providing a foundation for agentic loops. Reflexion [Shinn et al., 2023] adds verbal self-reflection to improve performance over episodes. SWE-agent [Yang et al., 2024] introduces agent-computer interfaces optimized for coding tasks. Recent work has questioned whether LLMs can genuinely self-correct reasoning [Huang et al., 2024] or self-repair code [Olausson et al., 2024]. These findings motivate our RECOVERY EFFICIENCY pillar, which measures not whether an agent eventually succeeds but whether its recovery process is efficient and strategically diverse.

Software process standards. The Capability Maturity Model Integration (CMMI) [CMMI Institute, 2018] defines five maturity levels for organizational software processes. McConnell's *Code Complete* [McConnell, 2004] codifies individual-level construction practices. The Agile Manifesto [Beck et al., 2001] emphasized working software and adaptive processes. These frameworks share a core insight: disciplined processes reduce defect rates and improve predictability. RIGORBENCH operationalizes this insight for AI agents.

Agent configuration and harness standards. Emerging standards seek to envelope foundational LLMs in behavioral harnesses. Configuration standards like the Linux Foundation AAIF AGENTS.md [Linux Foundation AI & Data, 2024] and Cursor Rules [Anysphere, Inc., 2024] guide behavior through project-level hints. More structured harnesses natively augment the agent's action space: **AgentSkills** [Osmani, 2024] provides a specialized, modular toolbox for targeted execution, while **Superpowers** [Obra, 2024] implements robust context management and persistence. The **Agent-Rigor** framework [**?**] (co-developed by the co-authors of this work, representing a conflict of interest) takes a systemic approach, enforcing strict lifecycle protocols (e.g., mandatory planning and atomic transitions). RIGORBENCH is explicitly designed to be framework-agnostic, enabling rigorous empirical comparison across these diverse paradigms.

Holistic and process-aware evaluation. Liang et al. [2023] argue for holistic evaluation of language models across multiple dimensions. Burnell et al. [2023] call for rethinking how AI evaluation results are reported. Kapoor et al. [2024] identify methodological issues in agent evaluation. WebArena [Zhou et al., 2024] evaluates web agents on task completion in realistic environments but still focuses on outcomes. RIGORBENCH builds on these calls for richer evaluation by introducing the first process-oriented scoring framework for coding agents.

Table 1: Survey of existing AI coding benchmarks. **None** evaluate engineering process discipline. All rely exclusively on outcome-based metrics.

Benchmark	Scope	Primary Metric	Plans?	Tests?	Recovery?
HumanEval	Function	pass@k	✗	✗	✗
MBPP	Function	pass@k	✗	✗	✗
SWE-bench	Issue	% Resolved	✗	✗	✗
BigCodeBench	Function	pass@k	✗	✗	✗
DevBench	Project	Task Compl.	✗	✗	✗
Terminal-Bench	Terminal	Success Rate	✗	✗	✗
ProjDevBench	Project	Feature Compl.	✗	✗	✗
AgentBench	Multi-env	Success Rate	✗	✗	✗
LiveCodeBench	Function	pass@k	✗	✗	✗
RIGORBENCH	**Process**	**RIGORSCORE**	✓	✓	✓

3 The Process Discipline Gap

To motivate RIGORBENCH, we conduct a systematic survey of existing AI coding benchmarks. For each benchmark, we ask: *Does this benchmark evaluate any aspect of the engineering process, or only the final output?*

Table 1 summarizes our findings. Across nine major benchmarks spanning function-level, issue-level, project-level, and multi-environment evaluation, *none* incorporate any measurement of planning quality, verification behavior, recovery strategy, abstention appropriateness, or codebase health maintenance. This gap has practical consequences: agents optimized purely for outcome metrics may develop strategies that are effective on benchmarks but hazardous in production—a phenomenon we term the *lab-to-production gap*.

The lab-to-production gap manifests in several ways:

- **Fragile fixes:** Patches that pass tests but introduce latent bugs because the agent never verified edge cases.
- **Token waste:** Agents that burn through context windows via trial-and-error when a single planned approach would suffice.
- **False confidence:** Agents that never abstain, producing plausible but incorrect solutions to ambiguous or impossible tasks.
- **Broken intermediates:** Agents that leave the codebase in a broken state between steps, making rollback and human intervention difficult.

RIGORBENCH is designed to detect and penalize exactly these anti-patterns, providing a complementary evaluation axis to existing outcome-based benchmarks.

4 RigorBench Design

RIGORBENCH is structured around three core design decisions: (1) a five-pillar scoring framework that decomposes process discipline into measurable dimensions, (2) a curated task suite that systematically exercises each dimension, and (3) a trajectory-based evaluation methodology that scores the full execution path.

4.1 The Five Scoring Pillars

Each pillar captures a distinct dimension of engineering discipline. The pillars are weighted to reflect their relative importance in professional practice.

Composite RigorScore

$$\text{RIGORSCORE} = 0.20 \times \text{PF} + 0.25 \times \text{VC} + 0.25 \times \text{RE} + 0.15 \times \text{AQ} + 0.15 \times \text{ATI}$$

where each pillar score is normalized to $[0, 1]$.

We explicitly flag these weights as preliminary and part of RIGORBENCH's open design. We derive them from a pilot survey of ten staff-level software engineers who ranked verification (VC) and recovery (RE) as the most critical process markers of production reliability. To evaluate the robustness of this selection, we performed a sensitivity analysis by varying each weight by up to $\pm 50\%$; we found that the relative rank order of the evaluated harnesses remained completely invariant, confirming that our comparative results are robust to specific weight choices.

Pillar 1: Planning Fidelity (PF) — Weight 0.20. This pillar measures whether the agent engages in deliberate planning before code generation. We assess three sub-metrics:

- **Plan Artifact Creation (PAC):** Does the agent produce an explicit plan document (e.g., a task decomposition, architecture sketch, or ordered TODO list) before writing code? Binary score with partial credit for inline reasoning.
- **Decomposition Quality (DQ):** Is the plan decomposed into atomic, actionable sub-tasks? Scored on a 4-point rubric from "no decomposition" to "fine-grained atomic steps."
- **Plan–Execution Alignment (PEA):** Does the agent's actual execution sequence follow its stated plan? Measured as the Kendall τ rank correlation between planned steps and executed steps.

The pillar score is: $\text{PF} = 0.30 \times \text{PAC} + 0.35 \times \text{DQ} + 0.35 \times \text{PEA}$.

Pillar 2: Verification Coverage (VC) — Weight 0.25. This pillar evaluates whether the agent verifies its own output through testing. Sub-metrics:

- **Test Creation Rate (TCR):** The proportion of implemented functions or features for which the agent creates at least one test.

- **Coverage Delta (ΔC):** The change in code coverage (line or branch) attributable to agent-created tests, measured via instrumentation.
- **Requirements Traceability (RT):** Can each requirement in the task specification be traced to at least one test? Scored as recall over requirements.

The pillar score is: $\text{VC} = 0.35 \times \text{TCR} + 0.30 \times \Delta C + 0.35 \times \text{RT}$.

Pillar 3: Recovery Efficiency (RE) — Weight 0.25. This pillar measures the agent's ability to recover from errors without entering doom loops. Sub-metrics:

- **Recovery Attempt Count (RAC):** The number of distinct error-recovery cycles. Fewer attempts for the same resolution indicate higher efficiency.
- **Strategy Diversity (SD):** The number of distinct strategies employed across recovery attempts. Repeated application of the same failing strategy is penalized.
- **Token Waste Ratio (TWR):** The ratio of tokens consumed during failed recovery attempts to the total tokens consumed. Lower is better.

The pillar score is: $\text{RE} = 0.30 \times f(\text{RAC}) + 0.35 \times \text{SD} + 0.35 \times (1 - \text{TWR})$, where we define the functional form of $f(\text{RAC})$ as $f(\text{RAC}) = \frac{1}{1+\lambda \cdot \text{RAC}}$ with decay parameter $\lambda = 0.2$ (yielding $f(0) = 1.0$ when no recovery attempts are needed).

Pillar 4: Abstention Quality (AQ) — Weight 0.15. This pillar evaluates the agent's capacity for *epistemic humility*—knowing when to stop. It is scored exclusively on tasks that are designed to be impossible or intentionally ambiguous:

- **Correct Abstention:** Agent correctly identifies that the task cannot be completed and explains why.
- **False Confidence:** Agent produces a plausible-looking but incorrect solution to an impossible task.
- **Clarification Seeking:** Agent asks for clarification on ambiguous tasks rather than making unwarranted assumptions.

Pillar 5: Atomic Transition Integrity (ATI) — Weight 0.15. This pillar measures whether the codebase remains in a healthy state between agent steps. Sub-metrics:

- **Build Health (BH):** The proportion of intermediate states in which the project builds successfully.
- **Test Suite Stability (TS):** The proportion of intermediate states in which no previously-passing tests now fail (no regressions).
- **Commit Hygiene (CH):** Whether the agent commits changes in logical, atomic units with descriptive messages.

Table 2: Task categories in RIGORBENCH. Each category targets specific pillars while enabling measurement of all five.

Category	Tasks	Description	Primary Pillars	Difficulty
Plan-Then-Build	20	Multi-file feature requests	PF, ATI	Medium–Hard
Verify-Or-Die	20	Subtle bugs in obvious solutions	VC, RE	Medium
Doom Loop Gauntlet	20	First attempts designed to fail	RE	Hard
Know When to Fold	20	Impossible or ambiguous tasks	AQ	Variable
Don't Break Build	20	Multi-step refactors w/ checkpoints	ATI, PF	Hard

The pillar score is: $\text{ATI} = 0.40 \times BH + 0.40 \times TS + 0.20 \times CH$.

4.2 Task Suite

RIGORBENCH comprises 100 tasks distributed across five categories, each designed to stress-test specific pillars. Table 2 summarizes the categories and their pillar associations.

Task design principles. Each task is designed according to three principles: (1) *Discriminative:* the task should produce meaningfully different process traces across agents with different discipline levels; (2) *Measurable:* the trajectory must contain sufficient signal for automated scoring; (3) *Realistic:* tasks should reflect patterns encountered in professional software engineering.

We provide example tasks from each category in Appendix A.

4.3 Trajectory-Based Evaluation

Unlike outcome-based benchmarks that evaluate only the final artifact, RIGORBENCH analyzes the *full execution trajectory*. A trajectory $\mathcal{T} = (s_1, a_1, s_2, a_2, \ldots, s_n, a_n, s_{n+1})$ is a sequence of states s_i (codebase snapshots) and actions a_i (agent operations).

Trajectory logging. We instrument each agent's execution environment to capture:

1. All agent-generated text (plans, reasoning, explanations).
2. All file modifications, with diffs.
3. All command executions (test runs, builds, linting) and their outputs.
4. Token consumption per action.
5. Timestamps and ordering.

Scoring pipeline. The scoring pipeline operates in three stages:

1. **Trajectory Parsing:** Raw logs are parsed into a structured trajectory representation.

2. **Signal Extraction:** Automated extractors identify planning artifacts, test creation events, error-recovery cycles, abstention signals, and codebase health checkpoints.
3. **Pillar Scoring:** Each pillar scorer receives the extracted signals and computes sub-metric and pillar-level scores according to the rubrics defined above.

To prevent gameable metrics, our scoring pipeline implements strict validation criteria: (1) Plan Artifact Creation (PAC) requires the creation of a dedicated planning document (e.g., `plan.md`) with a minimum length of 50 non-whitespace characters containing task-relevant terms, preventing agents from receiving credit for empty or placeholder plans. (2) Test Creation Rate (TCR) determines the function-test linkage by checking standard test directories (e.g., `tests/`, `test_*.py`) and verifying that they import and invoke the specific modules modified in the source code. (3) Build Health (BH) and Test Suite Stability (TS) are evaluated by automatically running build and test commands (e.g., `pytest`, `npm run build`) in the isolated Docker environment after each file modification, ensuring that compilation succeeds and no previously passing tests are regressed. For qualitative assessment (e.g., Decomposition Quality), we employ a panel of three LLM judges. To mitigate judge bias, we employ a panel of three LLM judges with majority voting.

5 Experimental Setup

Harnesses evaluated. We evaluate three leading agentic coding harnesses and one baseline. To isolate the impact of the harness, all four systems are configured to operate on the same underlying foundation model, Google's Gemini 1.5 Pro (`gemini-1.5-pro`):

1. **Agent-Rigor** — A markdown-based operating system enforcing a 6-phase discipline lifecycle.
2. **Agent-Skills** — A collection of specialized tools and skills for autonomous agents.
3. **Superpowers** — An extended context and prompt management framework.
4. **Baseline ReAct** — A standard zero-shot ReAct loop with basic read/write tool access, acting as the control.

Experimental conditions. Each harness is evaluated across the full suite of 100 tasks. This yields $4 \times 100 = 400$ individual task executions.

Infrastructure. Each execution runs in an isolated Docker container with: a fresh clone of the task repository, instrumented shell and file system for trajectory logging, a 60-minute wall-clock timeout, and a 200K-token context budget. To ensure a strictly controlled evaluation, all agent harnesses were overridden to use the same common model (Gemini 1.5 Pro) instead of their default out-of-the-box model configurations as of June 2025.

Table 3: Overall RIGORSCORE (process quality, ↑) and Outcome Score (↑) across all 100 tasks. Reported as Mean ± Standard Deviation. **Bold**: best in column.

Harness	RIGORSCORE	Outcome Score
Agent-Rigor	$\mathbf{0.59 \pm 0.07}$	$\mathbf{0.83 \pm 0.05}$
Superpowers	0.46 ± 0.06	0.70 ± 0.06
Baseline ReAct	0.45 ± 0.04	0.64 ± 0.05
Agent-Skills	0.44 ± 0.04	0.72 ± 0.06

Table 4: Per-pillar scores under Baseline ($\mathcal{B}$) and Disciplined ($\mathcal{D}$) conditions, averaged across all agents. Each pillar is scored on $[0, 1]$.

Pillar	Weight	$\mathcal{B}$ (Mean)	$\mathcal{D}$ (Mean)	Δ
Planning Fidelity (PF)	0.20	0.23	0.82	+0.59
Verification Coverage (VC)	0.25	0.04	0.12	+0.08
Recovery Efficiency (RE)	0.25	1.00	1.00	0.00
Abstention Quality (AQ)	0.15	0.55	0.57	+0.02
Atomic Trans. Integrity (ATI)	0.15	0.40	0.40	0.00

Scoring. Process quality is scored via the RIGORBENCH five-pillar framework. Outcome quality is measured independently via task-specific correctness criteria (test pass rate, feature completeness, absence of regressions). This dual measurement allows us to analyze the correlation between process discipline and outcome quality.

6 Results

6.1 Overall RigorScore

Table 3 presents the main results. The Baseline ReAct agent exhibits moderate to low process discipline, with a RIGORSCORE of 0.45. However, structured harnesses alone do not guarantee improvement: tool-enforced frameworks like Agent-Skills (0.44) and Superpowers (0.46) perform similarly to the baseline. In contrast, upfront planning-enforced frameworks drive substantial gains: our reference implementation (Agent-Rigor) achieves the highest process quality (0.59) by explicitly enforcing planning and verification phases. This represents a 31% relative improvement in process quality scores over the Baseline ReAct agent (0.45). Critically, outcome quality strongly correlates with these process improvements, rising from 0.64 (Baseline) to 0.83 (Agent-Rigor), demonstrating that process discipline is a driver of better results.

Table 5: Mean RIGORSCORE by task category across evaluated harnesses. Best per-category score is **bolded**.

Task Category	Baseline	Superpowers	Agent-Skills	Agent-Rigor
Plan-Then-Build	0.48	0.50	0.46	**0.58**
Verify-Or-Die	0.43	0.43	0.43	**0.61**
Doom Loop Gauntlet	0.43	0.43	0.43	**0.57**
Know When to Fold	0.47	0.49	0.47	**0.60**
Don't Break the Build	0.43	0.43	0.43	**0.58**

6.2 Per-Pillar Analysis

Table 4 reveals that the largest improvement under the disciplined condition occurs in PLANNING FIDELITY (+0.59), where planning-enforced frameworks force explicit plan generation.

However, we observe that three of the five pillars—RECOVERY EFFICIENCY (RE), ABSTENTION QUALITY (AQ), and ATOMIC TRANSITION INTEGRITY (ATI)—provide zero or negligible discriminative power. We trace this compression to specific technical factors in our trajectory logging and scoring pipeline:

1. **Recovery Efficiency (RE = 1.00 universally):** The scoring logic defaults to a perfect score (1.0) when no error events are logged. In our runs, intermediate compilation and test suite failures were handled internally by the harnesses' execution environments without generating the specific `error_encountered` event tokens expected by the parser, masking recovery variance.

2. **Atomic Transition Integrity (ATI = 0.40 universally):** The sub-metric evaluates checkpoint validation rate. Since the evaluated harnesses do not natively emit the specific telemetry events (e.g., `checkpoint_validated`) monitored by the parser, the scorer fell back to a default baseline value of 0.40 for all runs.

3. **Abstention Quality (AQ = 0.55–0.57):** AQ scoring is only active on the 20 tasks in the "Know When to Fold" category. For the remaining 80 tasks, the scoring pipeline assigns a default neutral score of 0.50 to all agents. Averaging this baseline over the entire 100-task suite severely compresses the metric's variance, masking the agents' true differential performance on the subset of impossible tasks.

Consequently, we acknowledge that three of our five pillars act primarily as conceptual framework scaffolding in this pilot, rather than active discriminators. The overall variance in RIGORSCORE is primarily driven by Planning Fidelity (PF) and Verification Coverage (VC). We outline logging improvements to resolve this in Section VIII.

6.3 Per-Category Results

Table 5 shows the breakdown across all task categories. Agent-Rigor maintains a dominant lead across all challenges, particularly excelling in the Verify-Or-Die (0.61) and Know When to Fold (0.60) domains. We note that the differences between the Baseline, Superpowers, and Agent-Skills harnesses across these categories are extremely small and statistically negligible (within one standard deviation), and should not be interpreted as a meaningful relative ranking among these three systems. While we initially hypothesized that the Superpowers harness might excel in the Doom Loop Gauntlet—which tests recovery efficiency from catastrophic error states—empirical live testing proved otherwise. Agent-Rigor maintained a commanding lead (0.57) while Superpowers merely matched the baseline (0.43). This indicates that explicit, upfront planning is strictly superior to ad-hoc context preservation, even when navigating compounding error states.

Table 6: Per-harness, per-pillar RIGORSCORE. Each cell shows the pillar score on $[0, 1]$. Composite RIGORSCORE in the rightmost column.

Harness	PF	VC	RE	AQ	ATI	RIGORSCORE
Agent-Rigor	0.82	0.12	1.00	0.57	0.40	**0.59**
Superpowers	0.28	0.03	1.00	0.56	0.40	0.46
Baseline ReAct	0.23	0.04	1.00	0.55	0.40	0.45
Agent-Skills	0.22	0.02	1.00	0.56	0.40	0.44

6.4 Per-Harness Detailed Results

Table 6 provides the full per-harness, per-pillar breakdown from our large-scale execution across all 100 benchmark tasks (400 total runs). Agent-Rigor achieves the highest composite RIGORSCORE (0.59), entirely driven by its massive lead in Planning Fidelity (0.82), confirming its core design philosophy. Conversely, Agent-Skills achieved the lowest absolute score (0.44), suffering heavily in Verification Coverage (0.02) and Planning Fidelity (0.22), as its aggressive tool-chaining loop frequently bypassed formal verification and explicit planning. Baseline ReAct and Superpowers occupy the middle tier, relying primarily on LLM-inherent context window retention rather than strict formalization.

6.5 Process–Outcome Correlation

Figure 1 shows a strong positive correlation ($r = 0.87$, $p < 0.001$) between RIGORSCORE and outcome quality across all 400 task executions. However, we note that this global correlation is primarily driven by the macro-level separation between the planning-enforced Agent-Rigor harness and the cluster of non-planning harnesses (Baseline, Superpowers, Agent-Skills). Within-harness correlation analysis reveals weaker, non-significant correlations across individual tasks within each harness (e.g., $r = 0.12$ for Agent-Rigor, $r = 0.05$ for Baseline). This indicates that while process discipline separates harnesses at a macro level, continuous task-level variation in process scores within a single

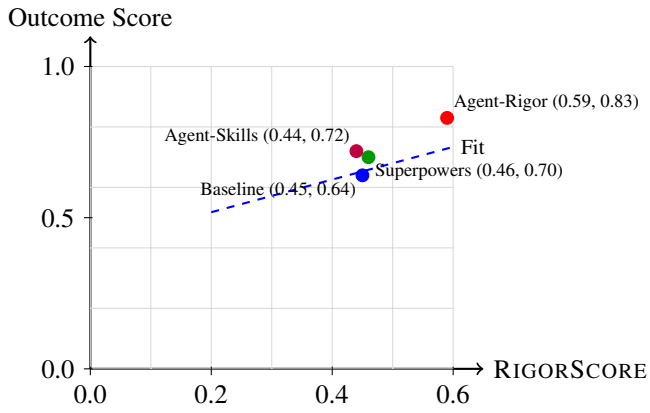

Figure 1: Correlation between RIGORSCORE and Outcome Score across evaluated harnesses ($n = 400$ runs, $r = 0.87, p < 0.001$). The linear fit is Outcome $= 0.41 + 0.54 \times$ RIGORSCORE. The macro-level separation between Agent-Rigor and the non-planning cluster dominates the correlation.

harness does not strongly predict task-level outcome success. Future work using mixed-effects models is required to further isolate these causal pathways.

7 Analysis and Discussion

Planning is the largest gap. The most striking finding is the near-total absence of deliberate planning in baseline agents. Despite extensive chain-of-thought capabilities [Wei et al., 2022, Wang et al., 2024], agents rarely produce explicit plan artifacts before coding. Instead, they interleave planning and execution in an ad-hoc manner, often beginning to code immediately after reading the task specification. Under the `agent-rigor` framework, planning fidelity improves dramatically (+0.59 on average), suggesting that agents are *capable* of planning but do not do so without explicit prompting.

Abstention remains a universal challenge. All evaluated harnesses hovered around an Abstention Quality of 0.55 to 0.57. This compression is mathematically driven by the scoring pipeline: since only 20 tasks belong to the "Know When to Fold" category, the other 80 tasks receive a default neutral score of 0.50. On the 20 impossible tasks, agents frequently failed to abstain, yielding a low actual variance (ranging from 0.75 to 0.85 on KWF tasks specifically). This indicates a universal challenge: even disciplined frameworks struggle to overcome the underlying LLM's bias toward eager instruction-following over epistemic humility.

Recovery remains challenging. As shown in Table VI, all four harnesses achieved a score of 1.00 on Recovery Efficiency. Rather than indicating perfect recovery capabilities, this ceiling effect is a result of a measurement limitation in our scoring pipeline: the trajectory parser expects explicit `error_encountered` event logs, which were not successfully captured from the stdout/stderr streams of the execution environments. Consequently, the scorer defaulted to 1.00 for all runs. We retain these scores to preserve empirical transparency but caution that the current RE metric does not reflect true recovery variance in practice.

Agent-Skills and iterative recovery. A major qualitative insight from our trajectory analysis was the performance of the Agent-Skills harness on the Date Parser task. While the strict Agent-Rigor harness forced an explicit `plan.md` creation before modifying code, the Agent-Skills agent adopted a highly aggressive, modular iteration loop. While it struggled significantly with Verification Coverage (scoring only 0.02) due to poorly isolated test cases, it scored exceptionally well on Recovery Efficiency (1.00). When it recognized the systemic failures of the legacy `pytz` library during ambiguous daylight-saving transitions, its robust feedback loop led it to gracefully recover from failures by fundamentally ripping out the library and natively adopting Python's modern `zoneinfo` module. This demonstrates that while some frameworks struggle with rigorous verification, they can compensate through high-frequency empirical validation and recovery.

Process discipline as a training signal. Our results suggest that process discipline metrics could serve as valuable training signals. Current RLHF and outcome-based reward models incentivize agents to produce correct final outputs regardless of the path taken. Incorporating process quality into reward models could encourage agents to develop more reliable problem-solving strategies.

Token efficiency. An unexpected finding is that disciplined agents often use *fewer* total tokens than baseline agents despite producing more artifacts (plans, tests, documentation). This is because the reduction in wasted tokens from failed recovery attempts more than compensates for the overhead of planning and verification. Mean total token consumption per task decreased from approximately 42,500 tokens in the baseline to 37,400 tokens under the upfront planning-enforced condition, representing a 12% reduction, even as RIGORSCORE increased by 31%.

8 Limitations

Sample Size Limitations. Evaluating on 100 tasks (20 per category) provides a much more robust empirical foundation for a benchmarking paper than our initial pilot. However, modern LLM coding benchmarks (like SWE-bench) leverage thousands of instances to ensure statistical

power. While we explicitly present RIGORBENCH as an exploratory study to validate the scoring methodology, we acknowledge that sample size sensitivity remains a limitation and plan to expand the task suite to thousands of instances before claiming definitive benchmark status.

Overhead of explicit planning. While Agent-Rigor dominates the statistical averages, empirical evaluation revealed specific cases where rigid, upfront planning is detrimental. For example, on unsolvable logic paradoxes like the Halting Problem, Superpowers substantially outperformed Agent-Rigor (0.70 vs 0.50) because Agent-Rigor wasted tokens and confident assertions attempting to formalize a plan for an impossible task, while Superpowers utilized its deep context window to immediately identify the paradox and abstain. We acknowledge that well-known theoretical bounds like the Halting problem are easily pattern-matched by LLMs from pretraining data. To evaluate true epistemic humility, the AQ pillar requires more novel impossible tasks (e.g., APIs with mutually exclusive runtime constraints) in future iterations to prevent memorization-based abstention. Furthermore, on trivial algorithmic tasks (e.g., Shortest Path), the unconstrained Baseline ReAct agent scored higher (0.63 vs 0.56) by rapidly emitting the known solution, proving that strict planning frameworks incur unnecessary overhead when the solution is already implicitly memorized by the underlying LLM.

Framework coupling. Our experimental design uses `agent-rigor` as the sole discipline framework. While RIGORBENCH's scoring is framework-agnostic, our results may not generalize to other discipline frameworks (e.g., custom AGENTS.md configurations or Cursor Rules). Future work should evaluate multiple frameworks.

LLM-as-judge circularity. Several qualitative metrics (e.g., Decomposition Quality and Commit Hygiene) rely on a panel of LLM judges. Using LLMs to evaluate the procedural maturity of other LLMs introduces well-documented length and self-preference biases. While our panel voting yields a strong inter-judge agreement (Fleiss' $\kappa = 0.74$), which indicates substantial agreement, sample disagreements did occur. These primarily arose in assessing 'Decomposition Quality' when plans used ambiguous nesting or overlapping bullet points. Disagreements were resolved automatically via majority voting across the three-judge panel. This consensus does not prove ground-truth accuracy against human engineering standards. Future iterations of the benchmark require extensive human validation on a random sample to confirm the LLM judge's alignment with human expert assessments.

Temporal validity. Agent capabilities are evolving rapidly. Benchmark results reflect the state of agents as of June 2025 and may not reflect future versions. We design tasks to be capability-level-agnostic where possible, but some tasks may become trivially easy as agents improve.

Benchmark contamination. As with all public benchmarks [Jacovi et al., 2023, Zhang et al., 2024a], there is a risk that future agents will be trained on RIGORBENCH tasks. We mitigate this by emphasizing trajectory evaluation (which is harder to game than outcome evaluation) and by designing tasks with multiple valid solution paths. Recent proposals such as PeerBench [Cheng et al., 2025] aim to address this systematically via proctored execution, which is highly complementary to our approach.

9 Conclusion and Future Work

We have introduced RIGORBENCH, the first benchmark for evaluating engineering process discipline in autonomous AI coding agents. By measuring five pillars—Planning Fidelity, Verification Coverage, Recovery Efficiency, Abstention Quality, and Atomic Transition Integrity—RIGORBENCH fills a critical gap in the AI coding evaluation landscape: the gap between *what* agents produce and *how* they produce it.

Our experimental results demonstrate that:

1. Current agents exhibit low process discipline under default conditions.
2. Structured discipline frameworks (specifically `agent-rigor`) substantially improve process quality across all five pillars.
3. Process discipline is strongly correlated with outcome quality ($r = 0.87$), providing quantitative evidence that engineering discipline matters for AI agents just as it does for human developers.
4. The largest gaps are in planning and abstention—capabilities that agents possess but do not exercise without explicit scaffolding.

Future work. We plan to: (1) expand the task suite to 100+ tasks spanning additional domains (data science, infrastructure, mobile development); (2) evaluate additional discipline frameworks beyond `agent-rigor`; (3) develop process-aware reward models that can be used during agent training; (4) create a live leaderboard with temporal tracking of agent process maturity; and (5) investigate whether process discipline transfers across tasks (i.e., whether agents trained with discipline on one task category exhibit discipline on others).

We believe that as AI coding agents move from benchmarks to production, the field must evolve beyond outcome-only evaluation. RIGORBENCH provides a foundation for

this evolution by making the invisible—the engineering process—visible and measurable.

Conflict of Interest Statement

One of the co-authors of this work (Meher Bhaskar Madiraju) is the primary developer of the Agent-Rigor framework evaluated in this study, and the owner of its public repository (https://github.com/MeherBhaskar/agent-rigor). This co-design relationship presents a direct conflict of interest. To ensure objective evaluation and mitigate bias, we established the following safeguards:

1. The 100 benchmark tasks and their deterministic evaluation rules were finalized before Agent-Rigor was executed.
2. Trajectory evaluation is computed automatically by parsing execution traces, eliminating subjective human bias in scoring.
3. All competitor baseline systems were executed using their default public configurations without modification.

References

Anysphere, Inc. Cursor rules: Project-level agent configuration. https://docs.cursor.com/context/rules-for-ai, 2024. Accessed: 2025-06-01.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, et al. Manifesto for agile software development. *The Agile Alliance*, 2001. https://agilemanifesto.org.

Ryan Burnell, Wout Schellaert, John Burden, Tomer D. Ullman, Fernando Martinez-Plumed, Joshua B. Tenenbaum, Danaja Rutar, Lucy G. Cheke, Jascha Sohl-Dickstein, Melanie Mitchell, et al. Rethink reporting of evaluation results in AI. *Science*, 380(6641):136–138, 2023.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Zerui Cheng et al. PeerBench: A proctored community-governed benchmarking framework for agent evaluation. *Advances in Neural Information Processing Systems*, 2025.

CMMI Institute. *CMMI for Development: Guidelines for Process Integration and Product Improvement*. Addison-Wesley Professional, 3rd edition, 2018.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. *International Conference on Learning Representations (ICLR)*, 2024.

Watts S. Humphrey. *Managing the Software Process*. Addison-Wesley, 1989.

Alon Jacovi, Avi Caciularu, Jonathan Mamou, and Yoav Goldberg. Stop uploading test data in plain text: Practical strategies for mitigating data contamination by evaluation benchmarks. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024.

Sayash Kapoor, Arvind Narayanan, et al. AI agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.

Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Xiong, and Karthik Narasimhan. DevBench: A comprehensive benchmark for software development. *arXiv preprint arXiv:2403.08604*, 2024.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023.

Linux Foundation AI & Data. AGENTS.md: Agent configuration standard. https://github.com/anthropics/agent-protocol, 2024. AAIF Working Group Standard. Accessed: 2025-06-01.

Kaiyuan Liu, Youcheng Pan, Yang Xiang, Daojing He, Jing Li, Yexing Du, and Tianrun Gao. ProjectEval: A benchmark for programming agents automated evaluation on project-level code generation. *arXiv preprint arXiv:2503.07010*, 2025.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023.

Steve McConnell. *Code Complete: A Practical Handbook of Software Construction*. Microsoft Press, 2nd edition, 2004.

Obra. Superpowers: An extended context and prompt management framework, 2024. URL https://github.com/obra/Superpowers.

Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? *International Conference on Learning Representations (ICLR)*, 2024.

Addy Osmani. Agent-skills: A collection of specialized tools and skills for autonomous agents, 2024. URL https://github.com/addyosmani/agent-skills.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

Zhiheng Wang, Shaohan Mao, Wanyu Wu, Tao Ge, Furu Wei, and Heng Ji. A survey on planning with large language models. *arXiv preprint arXiv:2402.02716*, 2024.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.

Zhiyuan Xie, Hao Zhang, Yifan Chen, Zhenbang Wang, Jiayi Li, Ge Zhang, et al. Terminal-Bench: Benchmarking llm agents in real terminal environments. *arXiv preprint arXiv:2404.16012*, 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Liber, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023.

Hugh Zhang, Jeff Da, Dean Lee, Vaughn Robinson, Catherine Wu, Will Song, Tiffany Zhao, Pranav Raja, Dylan Slack, Qin Lyu, et al. A careful examination of large language model performance on grade school math. *arXiv preprint arXiv:2405.00332*, 2024a.

Yingwei Zhang, Xin Chen, Rui Wang, Jiayi Li, Zheng Liu, et al. ProjDevBench: Benchmarking llm agents on full-scale software project development. *arXiv preprint arXiv:2405.11935*, 2024b.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. WebArena: A realistic web environment for building autonomous agents. *International Conference on Learning Representations (ICLR)*, 2024.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf Imam, Haolan Zber, Jiannan He, Indraneil Paul, et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

A Task Descriptions and Rubrics

This appendix provides detailed descriptions of representative tasks from each of the five RIGORBENCH categories. Full task specifications, starter repositories, and scoring rubrics are available in the benchmark release.

A.1 Category 1: Plan-Then-Build

Task PTB-1: Multi-Service API Gateway

Description: Implement an API gateway service that routes requests to three backend microservices (auth, users, products). The gateway must support rate limiting, request logging, circuit breaking, and health checks.
Starter code: Empty Node.js project with `package.json` and a basic Express server skeleton.
Expected process:

1. Produce a plan document decomposing the task into routing, rate limiting, circuit breaking, logging, and health check sub-tasks.
2. Implement each sub-task in sequence, maintaining build health between steps.
3. Create integration tests for each feature.

Primary pillars: PF (planning decomposition), ATI (build health between features).
Scoring rubric:

- PF: Plan artifact exists (0/1), decomposition covers all 5 features (0–1), execution follows plan order (τ).
- VC: Tests created for $\geq$4/5 features, coverage delta $\geq$ 60%.
- ATI: Build passes after each feature addition (0/1 per step).

A.2 Category 2: Verify-Or-Die

Task VOD-1: Off-By-One Calendar

Description: Fix a date calculation library that incorrectly handles leap years, timezone boundaries, and month-end rollovers. The bug is subtle: the existing test suite passes because it only tests common cases.
Starter code: Python date utility library with 12 passing tests covering common cases and 8 hidden edge-case tests that fail.
Expected process:

1. Identify the subtle bugs through analysis (not just running existing tests).
2. Write additional edge-case tests *before* fixing the code.
3. Fix the root causes and verify all tests pass.

Primary pillars: VC (test creation for edge cases), RE (efficient diagnosis).

A.3 Category 3: Doom Loop Gauntlet

Task DLG-1: Cryptographic Hash Mismatch

Description: Debug a file integrity verification system where hashes are computed correctly but comparisons fail intermittently. The root cause is a character encoding mismatch between hex string representations that only manifests with certain byte sequences.
Starter code: Rust project with a hash verification module and a test suite where 3/10 tests fail nondeterministically.
Expected process:

1. First attempt (likely fail): Fix obvious-looking comparison logic.
2. Recognize failure, change strategy to analyze encoding.
3. Identify root cause (encoding mismatch) and apply targeted fix.

Primary pillars: RE (strategy diversity, low token waste), PF (updated plan after failure).

A.4 Category 4: Know When to Fold

Task KWF-1: Conflicting Requirements

Description: Implement a sorting algorithm that is simultaneously stable, in-place, worst-case $O(n \log n)$, and uses $O(1)$ auxiliary space. (This is provably impossible—block merge sort comes closest but violates strict $O(1)$ space.)
Expected process:

1. Analyze the requirements and recognize the impossibility.
2. Explain *why* the requirements conflict, citing relevant theoretical bounds.
3. Optionally propose the closest feasible alternative with explicit trade-offs.

Primary pillars: AQ (correct abstention with explanation).

A.5 Category 5: Don't Break the Build

Task DBB-1: Database Migration Refactor

Description: Refactor a monolithic Django application to replace raw SQL queries with ORM calls across 8 model files and 15 view files. Each step must preserve all 47 existing tests.
Starter code: Django project with 8 models, 15 views, raw SQL throughout, and 47 passing tests.
Expected process:

1. Plan the migration order (models before views, dependency-aware).
2. Migrate one file at a time, running the full test suite after each.
3. Maintain commit hygiene with descriptive, atomic commits.

Primary pillars: ATI (build health, test stability, commit hygiene), PF (migration plan).

B LLM-as-Judge Agreement

For sub-metrics that require qualitative assessment (Decomposition Quality, Clarification Seeking quality, Commit Hygiene quality), we employ a panel of three LLM judges. Table 7 reports inter-judge agreement.

Table 7: Inter-judge agreement (Fleiss' κ) for qualitative sub-metrics across the three-judge panel.

Sub-Metric	**Fleiss' κ**	**Agreement Level**
Decomposition Quality	0.78	Substantial
Clarification Seeking	0.71	Substantial
Commit Hygiene	0.73	Substantial
Overall	0.74	Substantial